Independent Study Report

Xinwei Lin

May 17, 2013

1 Background

1.1 Motion Planning

Motion Planning is a term used for describing a process of navigating object movement through an abstracted network, by decomposing the whole problems into several smaller predefined motions. It is mostly used in robotics AI, specifically navigating mobile robots.

To solve a motion planning problem, a model needs to be built, converting real-life objects into data which can be presented in computers. A graph, or connected waypoints are the most common data structure to describe space in discrete planning, while as robots and obstacles come in different shapes and move in varied ways, a solid geographic representation must be formed. It covers both the geographic features and the kinematic features. Then, a proper motion planning algorithm needs to be introduced, in order to figure out a series of primitive motions which continually transform from one state to another, or to have it deemed impossible. These algorithms can vary from the simplest, i.e. single-direction forward search, to complicated ones.

There can also be uncertainty in motion planning. Uncertainty can come when the space is not fully explored, which leads to an incomplete or erroneous model, or as the robot moves, the space it moves in alter, which leads to an everchanging model, where every next step might need to reschedule according to the current environment. Unlike in static models, a decision must be made to determine what the next step is, which may require the assist of additional algorithm, in addition to the basic one.

1.2 Common Approach

Representation of Robots and Obstacles

Robots and obstacles take up spaces. They can get stuck in actual movement if simply abstracted as a point of no volume, so it is important to describe its surfaces in a model. This can be done with several methods.

The most intuitive one is to transform all their surfaces into continuous 2D polygons, then use the coordinates of the vertices forming up these polygons in an order relative to a point to represent the model. This method is likely to more accurately keep the original shape of the object, while its disadvantages are also obvious: when it serves as a collision detector, which is the reason why a geometric representation is needed, it takes up too much computation, as each face of the two possible collided object must be compared. It can be trimmed, in some ways, and some varied version based on this have been created to counteract this tradeoff.

Another one is more algebraic. A real model can be decomposed into several primitive shapes, such as spheres, pyramids and cuboids. The primitive shapes can be described in much fewer parameters. For example, consider a 3D space, a sphere can be defined by three for its center and one for its radius. Other shapes may need more, but it is still much fewer than the previous approach. The downside of this is that not every object can be easily decomposed into basic shapes, but in situations that not particularly strict collision avoidance is required, it is probably favorable to use this method. Also it greatly mitigates the difficulty in collision detection.

There are other methods, like using NURBS to describe their surfaces, especially when the surfaces are less flat. Bitmaps provide another possibility, but it is considered to cumbersome under certain circumstances.

Configuration Space

To describe the possible movements and positions a robot can be in, we can use configuration space. Considering a robot, in 3D space, where it can do translation along with any of the three axis or some of them combined, or rotation, which totals six degrees of freedom. Thus six parameters are required to describe the current state of the robot, besides its static shape. Each set of the parameters can be seen as a state, as with even one of the parameters changing, a new state is reached. The configuration space is such with the dimension of the degrees of freedom a robot can have, in which every vertex represents a state of the robot.

A robot can also have kinematic chains, which means part of the robot can have its own degree of movements. Consider a person waving his hands. Even if the person stands still, his hands have one degree of freedom (waving left and right, in this case), and such additional freedom contributes to the total degrees of freedom of a robot.

Some states can never been reached. Mostly it is because the collision of the robot and obstacles, or the robot against itself. These states can be precomputed, if in static model, and removed from the possible states. For the rest of the states, they are usually connected, meaning for each of the state, it is possible, through a basic transformation, to reach another state. If it is the case, these two states/vertices are defined connected. All the states that are connected group up a set.

The problem of motion planning in physical space can be transformed into finding a consecutive transformations from a source state to a sink state in configuration space.

Pathfinding

A great deal of motion planning algorithms exists to solve this. Some graph algorithms can be directly applied, or with some minor changes. We only talks about breadth first, random and Dijkstra's-like, as we only implement these for Demo #1.

Breadth First

The breadth first algorithm always choose the node which has the smallest amount of nodes in the path (depth) between the source and itself to expand. If we put all the nodes in the order of expansion in a FIFO queue, it is easy to show any node has less or equal depth than all of its subsequent nodes.

1. Initialize all nodes. Set all nodes UNVISITED except for the source being VISITED. Put source into FIFO queue.
2. Expand the first node in the FIFO queue. Put all its expansion into the tail of the queue. Dequeue this node.
3. Repeat step 2 until the sink node is in queue.

Random

The random algorithm randomly chooses a next node which can be expanded, and expand it.

1. Initialize all nodes. Set all nodes UNVISITED except for the source being VISITED.
2. Choose a random UNVISITED node which is directly connected to a VISITED node, expand it and set it VISITED.
3. Repeat step 2 until the current node is the sink node.

Dijkstra's-like

The original Dijkstra's algorithm is a classic graph search algorithm used to solve the shortest path problem between any two nodes. The classic form has one landmark restraints: the edges in the graph must all be non-negative.

1. Initialize all nodes. Set distance of the source node as 0 and others as infinite. Set the source node as current.
2. From current node, calculate the updated distance to all unvisited nodes. Expand the visited set by the unvisited node which has the shortest distance. Set this node as current.
3. Repeat step 2 until the current node is the sink node.

Its running time varies due to the data structure used. For sparse graphs, it works better with adjacency lists using binary heap or other similar data structure. It has a greedy algorithm's aspect in it - which means it always expands the node which seems the most favorable at the time of each step. It is one of the problems where greedy algorithms produce the optimal result, rather than an approximation, and it can be argued.

Some other algorithms, i.e. Bellman-Ford, can be seen as variations of Dijkstra's. A* is a typical generalization of Dijkstra's.

Others - A*

A* can be seen as a generalization of Dijkstra's. Instead of using the distance between a node to the source, an A* algorithm uses varied techniques to estimate a priority function F_i of each node i. F_i updates as the algorithm goes on and each step an extreme F_i is picked and gets expanded. The function F_i constructed greatly impacts the efficiency of an A* algorithm.

1.3 Subdivision and Soft Predicates

This section briefly reintroduces the some of the ideas we used in the demos, mostly introduced in *On Soft Predicates in Subdivision Motion Planning, Cong Wang, Yi-Jen Chiang and Chee Yap, 2013.*

Subdivision

The idea of subdivision method is to repetitively apply subdivision to the subdivided if necessary, repeat, and check each of the subdivided until we get one having certain properties we desire.

We define a box B as a set of points in configuration space. Commonly, it is always advisable to subdivide a box into as fewer children as possible, since the subdivision tree grows exponentially. In reality, we may consider dividing a box into 2^n children considering a quadrate/cubic box is often easier to compute its soft predicate (see below). We define a subdivision tree, where the root node is the whole configuration space (initial box). The children of a node is its subdivision.

Subdivision can go indefinitely if the subdivided show no properties we want, no matter how deep we get in. Thus we want to set up a condition to terminate if this is the case. Most intuitively, we can stop and backtrack if the depth of the subdivision tree reaches the height of n, and forbid the leaf nodes on level n to further subdivide regardless of the outcome.

Exact Predicates

Exact predicates are indicators of the state of a subdivision box describing whether it has collision against any of the obstacles. We define footprint F_p as a set of all the points in physical space within the robot when it is placed under configuration p. Also we define features as the abstracted types of obstacles which may stalls the movement of a robot. Features include corners (one or more dimensions), walls (two or more dimensions), faces (three or more dimensions) and more (in even

higher dimensions). It is easy to tell that a configuration p is impossible to place a robot on *iff* any of the features is within F_p. If this is the case, we call the exact predicate $C_p = STUCK$. If configuration p makes $F_p$slide through the edge of a feature, i.e. the feature is neither within nor outside of the robot, while other features are of the same situation or at least not within F_p, we call $C_p = MIXED$. Otherwise, we call $C_p = FREE$.

Similarly, we define $F_B = \cup F_p$, for $\forall p \in B$. We can also define and argue $C_B = \cup C_p$, where operator $\cup$ is defined in the following rules: $MIXED \cup any = MIXED$, $FREE \cup FREE = FREE$, $STUCK \cup STUCK = STUCK$, $FREE \cup STUCK = MIXED$. $\cup$ is both associative and commutative.

Soft Predicates

It is sometimes difficult as well as unnecessary to compute the exact predicates, which cost too much to compute. Soft predicates are introduced to replace exact predicates in the sense of:

1. It is faster to compute. Turn the problem of detecting collisions between polygons or polyhedrons into simpler problems like calculating distances. i.e. $cost(\tilde{C}_B) < cost(C_B)$.
2. It is safe. i.e. if $\tilde{C}_B \neq MIXED$, $\tilde{C}_B = C_B$.
3. It is convergent. i.e. for a series of boxes $\{B_i, i \in \{1, 2, ...\}\}$, if they converge into a configuration p, $\tilde{C}_{B_i} = C_p$ when i is large enough.

Same to exact predicates, soft predicates follow $\widetilde{C}_B = \cup \tilde{C}_p$. Using soft predicates allow much less computation to classify a single node, rendering it able to visit more nodes for a faster search routine.

Connectivity

Just like adjacent reachable configuration points are regarded to be mutually transformable, if a box is B which is$FREE$, it essentially means $\forall p_1, p_2 \in B$, there can be found a series of transformation from $p_1$to p_2. We consider two boxes B_1, $B_2$to be connected provided

1. .$C_{B_1} = FREE, C_{B_2} = FREE$
2. $\exists p_1 \in B_1, \exists p_2 \in B_2$, where $p_1$and p_2 are adjacent.

In order to apply pathfinding algorithms on our model, it needs to convert to a graph. We do the following:

- Take each box as a node.
- For each pair of two connected boxes, there exists a bidirectional edge.
- Each time a box is to be subdivided, the corresponding node is split and the graph is partially reconstructed. Recalculate the connectivity among the sub-nodes, as well as the connectivity between each of sub-nodes to any of the other nodes connected to the node split. Varied methods exist to simply the calculation.

2 Demo #1: Motion Planning of 3D-ball, translation only

This demo serves as a 3D adaption of the existing demo: visualization of the motion planning of a disc through a set of obstacles in 2D-space.

2.1 Algorithm

The algorithm resembles which of the 2d-disc, with tweaks for better visualization and considering the more complicated scenario in 3D.

The algorithm tries to find a path connecting the source configuration to the sink configuration. More precisely, it tries to find a series of continuous FREE boxes where the first of the boxes includes the source and the last of the boxes includes the sink. To do this, we must know how to classify a box, how to subdivide a box if it is MIXED and how to find a path provided we know how to do the previous two.

We define robot $R(r_{sphere})$ box $B = (x_B, y_B, z_B, r_B)$where it encompasses all the points within a cube in configuration space, with the center of which being $(x_B, y_B, z_B)$and the radius being r_B. Coincidentally, in this setting each of the points in configuration space corresponds to the center of the sphere in physical space of the same coordinates. For example, $P(x_B, y_B, z_B)$ in configuration space corresponds to the footprint $F(x_B, y_B, z_B) = \{\forall(x, y, z)|\sqrt{(x - x_B)^2 + (y - y_B)^2 + (z - z_B)^2}\leq r_{sphere}\}$.

Classification

We use the soft predicates (see 1.3) and modified the algorithm presented in 2d-disc. Consider box B, it can be argued that $F_B \subseteq \{\forall(x, y, z)|x_B - r_{sphere} - r_B \leq x \leq x_B + r_{sphere} + r_B, y_B - r_{sphere} - r_B \leq y \leq y_B + r_{sphere} + r_B, z_B - r_{sphere} - r_B \leq z \leq z_B + r_{sphere} + r_B\} \subseteq \{\forall(x, y, z)|\sqrt{(x - x_B)^2 + (y - y_B)^2 + (z - z_B)^2}\leq r_{sphere} + r_B\}$. This means any feature which has a distance to point P more than $r_{sphere} + r_B$ will not collide with the footprint F_B. If all the features don't collide with F_B, we can safely say the soft predicate of the box $\tilde{C}_B = FREE$.

In the other way around, it can also be argued that $\cap F_{(x,y,z)\subseteq B} \supseteq \{\forall(x, y, z)|(x - x_B)^2 + (y - y_B)^2 + (z - z_B)^2\leq(r_{sphere} - r_B)^2\}$. if $r_{sphere} \geq r_B$, any feature which has a distance to point P less or equal than $r_{sphere} - r_B$ will collide with the footprint of any configuration of robot which locates within B. If this is true, we can safely say the soft predicate of the box $\tilde{C}_B = STUCK$.

Any box which does not satisfy any of the previous condition are considered $\tilde{C}_B = MIXED$. Any feature that potentially makes this box not $FREE$ should be inherited to its children if it is to be divided.

Distance/Separation

The distance between P to a corner (point) is trivial. The distance between P to a line segment can be computed as following: if the projection of the point falls within the line segment, the distance is the distance between P and the infinite line which the line segment is in. Otherwise, the distance equals the shorter distance between P and either of the two ends. The distance between P to a triangle is a bit more complicated, please refer to pointTriangleDistance.m attached in the subdirectory /K_3/K_3.

Subdivision

Similar to 2d-disc, we can subdivide a box into eight boxes of the same radius. Consider a box $B_0 = (x, y, z, r)$, it will be divided into the following boxes:

$B_1 = (x + r, y + r, z + r, r/2),\ B_2 = (x + r, y + r, z - r, r/2)$

$B_3 = (x + r, y - r, z + r, r/2),\ B_4 = (x + r, y - r, z - r, r/2)$

$B_5 = (x - r, y + r, z + r, r/2),\ B_6 = (x - r, y + r, z - r, r/2)$

$B_7 = (x - r, y - r, z + r, r/2),\ B_8 = (x - r, y - r, z - r, r/2)$

Only MIXED boxes are considered worth subdividing. If a box is either FREE of STUCK, no subdivision is needed as even if we subdivide them, their children will be all FREE or STUCK (see 1.3).

Also, to avoid subdivision going too deep, we truncate the subdivision tree when the radius of a leaf box is equal or less than a given value ε.

Pathfinding

First, we want to place the source configuration into a FREE box. This can be achieved by keeping subdividing the initial box following the rule mentioned above. If we cannot get a FREE box in this process, the algorithm halts and no path is found.

Then, we use either of the three methods (see 1.2) to expand the set of VISITED nodes, with the modification of:

- Never expand a node indicating a STUCK box.
- Do not expand a node indicating a MIXED box (MIXED node) directly. Instead, subdivide this node and replace it with its eight children. Recompute whether it is directly accessible from each of the children to each of its connected node.

Extra Check: Position of the initial position

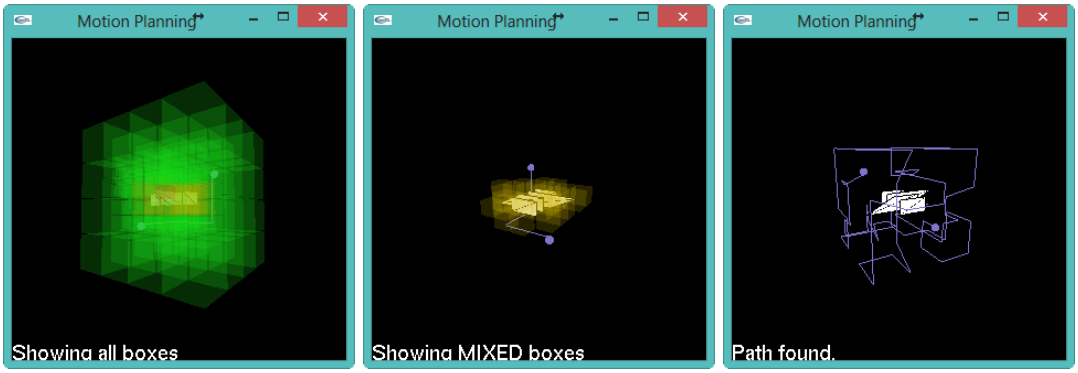
Although the previous described algorithm excludes the possibility to move from a location within obstacles to a location outside, or other way around, but the source and the sink configuration may signify a position rendering a robot within an obstacle. If it is the case, there must be no path available. We can check this condition prior to the subdivision of the boxes to optimize computation.

2.2 Functionality

The demo includes a main window along with a simulated command line tool. The main window visualize the configuration space in 3D. You can interact with the application through the following ways.

- Mouse. Drag to rotate the view point around the center of the initial box.
- Keyboard: Shortcuts to zoom in and out, toggle display the type of boxes (FREE/MIXED/STUCK), path, robot and obstacles, and run the breadth first algorithm.
- Command line: All of the above, run any of the three implemented pathfinding algorithms, and load robot/obstacle files.

Please refer to README for detailed instructions.


Motion Planning
Showing all boxes
Motion Planning
Showing MIXED boxes
Motion Planning
Path found.

3 Demo #2: Mapping 2-sphere onto 3-cube

This demo aims at the visualization of the mapping between two forms of representation of two degrees of rotations, (θ, φ) and (x, y, z).

3.1 Algorithm

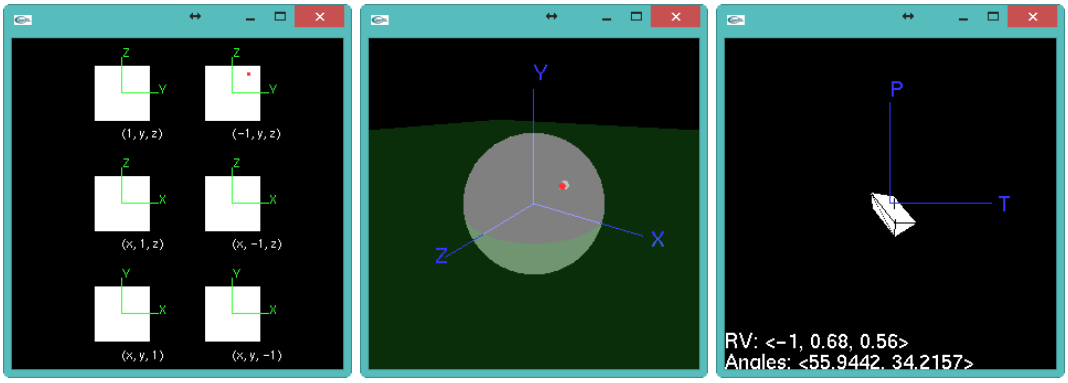
Consider a sphere $S = \{\forall(x, y, z)|\sqrt{x^2 + y^2 + z^2} = 1\}$ and its circumscribed cube $C = \{\forall(x, y, 1)| - 1 \leq x \leq 1, -1 \leq y \leq 1\} \bigcup \{\forall(x, 1, z)| - 1 \leq x \leq 1, -1 \leq z \leq 1\} \bigcup \{\forall(1, y, z)| - 1 \leq y \leq 1, -1 \leq z \leq 1\}$. Each ray emitted from the origin will pass through S and subsequently C. We map the intersection on S to the intersection on C.

We can alternatively represent $S = \{\forall(\theta, \varphi)|\theta \in [0, 2\pi), \varphi \in [-\pi/2, \pi/2]\}$. We can regard θ as the longitude of S and φas the latitude of S. Each of the rays can be thus defined $l(\theta, \varphi) = \{(ksin\theta, kcos\theta, ksin\varphi)|k \geq 0\}$. $l(\theta, \varphi)$ intersects S at $(\frac{sin\theta}{\sqrt{sin^2\varphi+1}}, \frac{cos\theta}{\sqrt{sin^2\varphi+1}}, \frac{sin\varphi}{\sqrt{sin^2\varphi+1}})$ and the six 2-cubes or their extension planes on $(1, tan^{-1}\theta, \frac{sin\varphi}{sin\theta})$, $(tan\theta, 1, \frac{sin\varphi}{cos\theta})$, $(\frac{sin\theta}{sin\varphi}, \frac{cos\theta}{sin\varphi}, 1)$ if applicable. The conversion back is similar.

3.2 Functionality

The demo consists of three views and a control panel. All three views are synchronized and a change on current configuration will update all three views.

- Object view: This view provides the visualization of a custom model rotating around X-axis and Z-axis. Can interact with mouse rotate the object.
- 2-Sphere view: This view provides the visualization of the 2-sphere and the corresponding point in configuration space lighted in red. Can interact with mouse to change the viewport.
- 3-Cube view: This view dissect the 3-cube into its six 2-Cube and shows the corresponding point in configuration space lighted in red. Can interact with mouse to drag and click to change the current configuration.
- Control Panel: Can reset, restart and quit the demo. Can also change the model of the custom object using an existing model file.



4 Demo #3: Feature Detection of 3D-cigar, two degrees of freedom of rotations only

This demo will illustrate the features that may collide with a cigar and/or a ring in a specific subdivision box. It is currently a work-in-progress.

4.1 Algorithm

Connectivity

Compared to subdividing translation-only configuration space, the topology of the rotation part is a bit different. Consider in the configuration space with only one degree of freedom of rotation$\theta \in [0, 2\pi)$. When split into two, i.e. $[0, \pi)$ and $[\pi, 2\pi)$, they connect not only on $\theta = \pi$, but also $\theta = 0$. Such feature impacts the connectivity when transforming into a graph.

Exact Predicates of a static Cigar Robot

A cigar can be segregated into three sections: The spine (with centerline AB), a circular cylinder; the apex (with center A), which indicates the top sphere where the center locates at the center of the top face of the spine with a radius equivalent to which of the spine; and the base (with center B), which indicates the bottom sphere where the center locates at the center of the bottom face of the spine with a radius equivalent to which of the spine. The two planes which intersect A and B which the great circle which is perpetual to the spine cut the physical space into three regions: AB-region where the spine is in, A-region which contains the upper hemisphere of the apex, and B-region which contains the lower hemisphere of the base. We define such cigar as $c(r, l)$ where r is the radius and l is the length of the cylinder.

A 3D-cigar can rotate at three degree of freedom of rotation, but if we align one of the orthogonal axises, rotation around this axis will not affect the footprint of this sets of configuration, i.e. $F_{(x,y,z_1)} = F_{(x,y,z_2)}, \forall z1, z2 \in [0, 2\pi)$. As the collision detection only involves consideration of whether there exists any feature which may intersect the footprint, we reduce the rotation to two degrees of freedom, and the rotation vector to (x, y).

For each of the features that may collide with the cigar, first we check whether it falls into any of the three regions. For those features which cross more than one region, we consider them falling into all of those regions. The exact predicates of each regions are defined as:

1. A-region. It is essentially a hemisphere, thus it is purely related to the distance between the center of the apex and the feature. (See 2.1 distance/separation)

2. B-region. De facto the same to A-region.

3. AB-region. Since the feature does not locate above or below the cylinder (even if this is not the case, it is already discussed in the previous conditions). We only need to consider the distance between the centerline and the feature. Doing so involves computing the distance between a line segment and points, other line segments and triangles.

We define the shortest distance of a certain feature f when the robot is placed under a configuration v above as $d_{v,f}$. Thus $C_v = STUCK$ *iff* $d_v = min(d_{v,f})$, for f in all features.

Soft Predicates of a rotation box of a cigar robot

onsider a subdivision box covering an interval of (θ, θ). We define all the boxes with the same interval as B_θ.

- In A-region, If θ is small enough, the footprint of point A is roughly a circle with radius of $\frac{l_0}{2} sin\theta \approx \frac{l_0}{2}\theta$. Thus the furthest point in the footprint of the apex to A will not exceed $d_{B_\theta}(A) = r_0 + \frac{l_0}{2}\theta$. If a feature f is in A-region and its distance to A is more than $d_{B_\theta}(A)$, we can safely say it is $SAFE$. Applying the technique introduced in demo #2, we can subdivide the six 2-cube much more easily than a 2-sphere. We define a subdivision box covering an interval of (r, r) on a 2-cube B_r. If we project a set ofB_θ to 2-cubes, we find the further it deviates from the center of the 2-cube, the more sparse it becomes. i.e. At the corner, it needs the biggest B_{corner} to cover the projection. We choose this box as an extrenum, thus $\theta \leq arctan(r)$. Putting it together, if $d_{B_\theta, f}(A) > r_0 + \frac{l_0}{2} arctan(r), \forall f \in A - region$, $\tilde{C}(A) = FREE$. Otherwise $\tilde{C}(A) = MIXED$.
- B-region is de facto the same as A-region.
- AB-region. Still a work-in-progress.

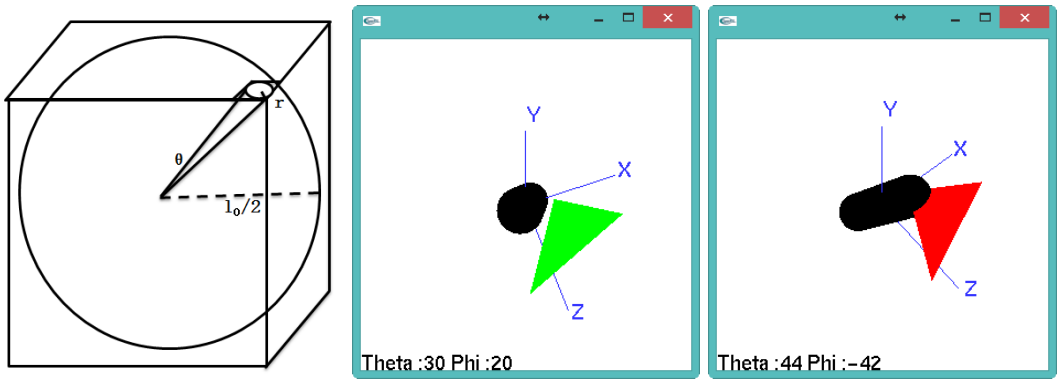
4.2 Extension to $SO(2) \times \mathbb{R}^3$

When combining translation and rotation together, we want to subdivide translation first, and rotation later. It is because we know the soft predicates of translation better than which of rotation. We may practice one of the following strategies:

- Always subdivide translation part first. Subdivide rotation only until ε_t is reached.
- Subdivide translation part first, until it reaches ε_{bigger}, which generally bigger than ε_t, then split rotation.

4.3 Expected functionality

A cigar in rotation, lighting up with the features which the exact predicates (of current rotation vector) or soft predicates (if showing a level of subdivision box where the current rotation vector lies in) deems possible to intersect. Use mouse to rotate the viewport.



5 Future Work: Migration to web page

Here are some of my experimental ideas and surveys of migrating existing and future demos onto a webpage.

5.1 Demos using Java awt/swing

It is rather simple to migrate a Java desktop application onto a java applet, which can be later embedded into a HTML page. The detailed information can be found in http://download.oracle.com/javase/tutorial/deployment/applet/deployingApplet.html. Briefly the steps are:

- Extend the main class with JApplet.
- Compile and pack it to a *.jar file.
- Add script to specify both the location of the *.jar file and the main class.

Limitations

Currently the Java 7 only works with 64-bit applications. It means certain browsers cannot view the applets if it is 32-bit based, i.e. Chrome. To enable Java Applets on Chrome, the applets must be compiled in Java 6, which may disable certain features newly introduced in Java 7. We can upload both of the versions and only show the users the version their browser is complied with to provide best viewing experience.

5.2 Demos using OpenGL

A similar technology, WebGL, is developed based on OpenGL 2.0. It runs on JavaScript and is compatible with almost all modern browsers. To convert C/C++ code using OpenGL to WebGL, we must:

- Separate logic/algorithm and visualization. Most of the existing demos congregate these two, as the algorithm runs within the rendering function of OpenGL rather than separately on another thread. The difficulty of doing so varies depending how the code is constructed.
- The logic/algorithm runs on the server side. This does not need to change.
- When the client requests the algorithm to be run, the server will run it and push back the data after it finishes computation.
- The visualization code is purely on the client side. After receiving all the data, it draws on the browser using WebGL.

Challenges

Unlike OpenGL, which has several utility wrapper library on top of it, i.e. GLU, GLUT, GLUI, which means all of the function calls to these libraries must be manually rewritten. Also because JavaScript is analyzed on client's end, sadly there can not be any library assisting this effort.

It is also noticeable that the lack of GLUI enforces us to switch to other types of control manager.

Although most of our demos are not visualization-intensive, but those with animations can prove to be quite hard to convert, in my opinion.